

Recurrence-Free Closed Forms for Dyadic Fabius and Rvachev Values

Positive compositions, cumulant partitions, binary compression,
finite uniform prefixes, and quarter-base q -filters

Research article prepared for Vladimir Reshetnikov

4 September 2026

Abstract

Exact rational evaluation of the Fabius function at a dyadic argument is often presented using a recurrence for moments or for values at reciprocal powers of two. This article removes those recurrences completely. It develops positive ordered-composition sums, Bernoulli and Bernoulli-free multiplicity-partition sums, finite-uniform-prefix formulas, and an integer bordered-determinant formula with an explicit permutation interpretation. A binary-block identity reduces the argument-dependent part of the evaluation from a Thue–Morse prefix of length a to at most $n + 1$ terms for the argument $a/2^n$, including the signed global extension. A second, moment-free answer is a finite spline formula with Gaussian base $1/4$; it uses only $\lfloor n/2 \rfloor + 1$ prefixes and has no limiting operation. Its exactness is proved from polynomial pieces of finite splines, not from an invalid assumption that the Fabius function equals its finite Taylor jet. All evaluation formulas involve only explicitly bounded finite sums, products, factorials, integer powers, binary digits, and optional q -Pochhammer symbols. Proofs, normalization checks, exact examples, and a reproducible rational-arithmetic verification program are included.

Scope. The bounded Fabius distribution, the compactly supported Rvachev bump, and the signed global Fabius extension are kept distinct throughout. The results are mathematical derivations; no new Lean verification is claimed.

Contents

1	The question and the main answers	3
1.1	A guide to the formulas	3
1.2	Conventions	3
2	Normalization and the analytic identities used in the proofs	4
2.1	Three related functions	4
2.2	Density, smoothness, and dilation	4
2.3	Normalized moments and their generating function	5
3	The universal finite dyadic reduction	5
4	Positive ordered-composition formulas	5
5	Multiplicity partitions and explicit cumulants	6
5.1	A compact partition formula	6
5.2	Removing even the Bernoulli symbols	7
5.3	Why this is a different useful representation	8
6	Binary compression of the argument-dependent sum	8
6.1	One formula with no unresolved coefficient symbols	9
7	Finite uniform prefixes and a moment-free multinomial formula	9
7.1	The explicit finite-coordinate polynomial	9
7.2	Explicit Gaussian weights	10
7.3	Arbitrary finite prefixes and other bases	11
8	An exact quarter-base spline formula with no moments	11
8.1	The finite spline in explicit form	11
8.2	Why the scale dependence is a finite polynomial	12
8.3	The closed double sum	13
8.4	Other prefix depths, conditioning, and the half-base comparison	14
8.5	When the degree is sharp	14
9	An integer determinant and its explicit permutation sum	15
10	Domains, normalizations, and dyadic derivatives	16
10.1	Bounded Fabius and compact Rvachev values	16
10.2	Signed global values and nonreduced fractions	16
10.3	Closed forms for every dyadic derivative	17
11	Exact examples and independent arithmetic checks	17
11.1	Universal coefficients and a dyadic table	17
11.2	Two-point and three-point spline extraction	18
11.3	Examples with small values and larger denominators	18
11.4	What was checked computationally	19
12	How the alternatives compare	19
12.1	Closed form versus evaluation algorithm	19
12.2	Relation to the supplied repository and the literature	20
13	Conclusion	20
A	A compact nonrecursive exact evaluator	20
B	Notation and reproducibility checklist	22

1 The question and the main answers

A finite formula that refers to “the moments c_k , computed recursively” is not a closed form in the sense used here. Neither is a limit of explicit rational approximations. The desired endpoint is an expression in which every auxiliary quantity itself has a finite, nonrecursive definition.

The reference exposition in the **ProveIt** repository already contains the Thue–Morse moment formula, reciprocal-power path and composition descriptions, Bell/cumulant machinery, and half-base Gaussian identities [1]. Arias de Reyna gives the classical dyadic moment formula in his arithmetic paper [2]; the earlier paper supplies its analytic setting [3]. Reshetnikov’s q -binomial question [4] and Pinelis’s finite-convolution construction [5] provide useful comparisons. The aim below is not to claim priority for these methods, but to derive alternative, fully expanded evaluation formulas and show exactly why they terminate.

1.1 A guide to the formulas

There are two particularly useful answers. The first combines the binary formula (27) with either the positive composition coefficients (16) or the partition coefficients (20). Its argument-dependent sum has at most $n + 1$ nonzero positions, even when the numerator is very large. Equation (31) writes this answer without leaving any moment symbol unresolved.

The second answer is the double sum (47). It contains no moment, cumulant, or derivative symbols at all. Its outer sum has only $d + 1$ terms, where $d = \lfloor n/2 \rfloor$, and its Gaussian base is $1/4$. It is exact, not an extrapolation limit.

Two further representations emphasize different algebra. Equation (37) uses finitely many uniform coordinates and a finite multinomial sum. Equation (57) uses a single integer determinant; (58) explicitly expands that determinant as an allowed finite sum of products. None requires an auxiliary recurrence.

1.2 Conventions

Throughout, n, a are nonnegative integers unless a different range is stated, and $\mathbb{N} = \{0, 1, 2, \dots\}$. Write

$$\tau_n = \binom{n+1}{2}, \quad d = \lfloor n/2 \rfloor, \quad \varepsilon_h = (-1)^{s_2(h)}, \quad (1)$$

where $s_2(h)$ is the number of ones in the binary representation of h . This is a digit formula, not a recursively defined sequence. Empty sums are zero, empty products are one, and $0^0 = 1$ in the finite polynomial formulas.

For $k \geq 0$, let $\mathcal{C}_k$ be the finite set of all ordered compositions of k into positive parts, with the empty composition admitted only for $k = 0$. For $\mathbf{r} = (r_1, \dots, r_\ell) \in \mathcal{C}_k$, put

$$R_j = r_j + r_{j+1} + \dots + r_\ell. \quad (2)$$

The finite multiplicity-profile set is

$$\mathcal{P}_k = \{(m_1, \dots, m_k) \in \mathbb{N}^k : m_1 + 2m_2 + \dots + km_k = k\}. \quad (3)$$

For $k = 0$ this consists of one empty tuple. Every definition in (1)–(3) is explicit.

2 Normalization and the analytic identities used in the proofs

2.1 Three related functions

Let $U_1, U_2, \dots$ be independent uniform random variables on $[0, 1]$, and set

$$X = \sum_{j=1}^{\infty} 2^{-j} U_j, \quad F(x) = \mathbb{P}(X \leq x). \quad (4)$$

Thus $F(x) = 0$ for $x \leq 0$, and $F(x) = 1$ for $x \geq 1$. Reflection of every coordinate gives $F(1-x) = 1 - F(x)$. The Rvachev normalization is

$$\text{up}(y) = F(1 - |y|), \quad (5)$$

so up is even, supported on $[-1, 1]$, and $\text{up}(0) = 1$. The signed global extension is

$$\Phi(x) = \sum_{b=0}^{\infty} \varepsilon_b \text{up}(x - 2b - 1). \quad (6)$$

Only one summand can be nonzero at a given point. In particular,

$$\Phi(x) = \varepsilon_b \text{up}(x - 2b - 1) \quad (2b \leq x \leq 2b + 2), \quad \Phi = F \quad \text{on } [0, 1]. \quad (7)$$

The function Φ , not the bounded F , satisfies the pure dilation equation on the whole real line.

2.2 Density, smoothness, and dilation

Put $V_j = 2U_j - 1$. The centered random variable

$$Y = 2X - 1 = \sum_{j=1}^{\infty} 2^{-j} V_j \quad (8)$$

has density up . Here is a direct normalization check. The law of Y is that of $(V + Y')/2$, with V uniform on $[-1, 1]$ and Y' an independent copy of Y . If H denotes its distribution function, its density is

$$H(2y + 1) - H(2y - 1).$$

For $0 \leq y \leq 1$, this equals $1 - F(y) = F(1 - y)$; for $-1 \leq y \leq 0$, it equals $F(1 + y)$. Outside the support it vanishes. Convolution with the first two uniform coordinates gives a continuous density. The displayed identity then differentiates to

$$\text{up}'(y) = 2 \text{up}(2y + 1) - 2 \text{up}(2y - 1), \quad F'(x) = 2 \text{up}(2x - 1). \quad (9)$$

Repeated differentiation bootstraps continuity to C^∞ . Compact support implies flatness at the endpoints of up , and hence flatness of Φ at zero. Substitution of (9) into (6), with the even and odd binary signs separated, gives

$$\Phi'(x) = 2\Phi(2x), \quad \Phi^{(r)}(x) = 2^r \Phi(2^r x) \quad (r \geq 0). \quad (10)$$

These arguments also show why the Fourier normalization is

$$\widehat{\text{up}}(\xi) = \prod_{j=1}^{\infty} \frac{\sin(2\pi\xi/2^j)}{2\pi\xi/2^j} = \prod_{k=0}^{\infty} \frac{\sin(\pi\xi/2^k)}{\pi\xi/2^k}, \quad \widehat{f}(\xi) = \int_{\mathbb{R}} f(y) e^{-2\pi i \xi y} dy. \quad (11)$$

No factor of two is absorbed into the definition of up .

2.3 Normalized moments and their generating function

Define

$$c_k = \mathbb{E}Y^{2k}, \quad b_k = \frac{c_k}{(2k)!}, \quad M(t) = \mathbb{E}e^{tY} = \sum_{k=0}^{\infty} b_k t^{2k}. \quad (12)$$

The support bound $|Y| \leq 1$ makes M entire. Independence gives

$$M(t) = \prod_{j=1}^{\infty} \frac{\sinh(t/2^j)}{t/2^j}, \quad M(2t) = \frac{\sinh t}{t} M(t). \quad (13)$$

The product converges locally uniformly, since its factors are $1 + O(4^{-j}|t|^2)$ on bounded sets. These analytic expressions will be used to *prove* finite identities. They are not required in any final evaluation.

3 The universal finite dyadic reduction

Theorem 3.1 (Moment-to-value transfer). *For every $a, n \geq 0$,*

$$\Phi\left(\frac{a}{2^n}\right) = 2^{-\tau_n} \sum_{h=0}^{a-1} \varepsilon_h \sum_{k=0}^{\lfloor n/2 \rfloor} b_k \frac{(2a - 2h - 1)^{n-2k}}{(n - 2k)!}. \quad (14)$$

For $0 \leq a \leq 2^n$, the left side is $F(a/2^n)$.

Proof. Taylor's integral formula at the flat point zero, applied with order $n + 1$, gives

$$\Phi(x) = \frac{1}{n!} \int_0^x (x - t)^n \Phi^{(n+1)}(t) dt.$$

Using (10) and changing variable $v = 2^{n+1}t$ yields

$$\Phi(x) = \frac{2^{-\tau_n}}{n!} \int_0^{2^{n+1}x} (2^{n+1}x - v)^n \Phi(v) dv. \quad (15)$$

At $x = a/2^n$, the integration interval is exactly $[0, 2a]$, a union of a complete signed bump blocks. Substitute $v = 2h + 1 + y$ on each block:

$$\Phi\left(\frac{a}{2^n}\right) = \frac{2^{-\tau_n}}{n!} \sum_{h=0}^{a-1} \varepsilon_h \int_{-1}^1 (2a - 2h - 1 - y)^n \text{up}(y) dy.$$

Odd moments vanish. Expanding the power and using $\binom{n}{2k} c_k / n! = b_k / (n - 2k)!$ proves the formula. The same argument includes $n = 0$; an empty integration interval covers $a = 0$. $\square$

This is the familiar dyadic moment identity in normalized coefficients. Its value for the present problem is that only $b_0, \dots, b_d$ occur. The next sections replace every one of these coefficients by a finite formula.

4 Positive ordered-composition formulas

Theorem 4.1 (Positive coefficient formula). *For every $k \geq 0$,*

$$b_k = \sum_{\mathbf{r} \in \mathcal{C}_k} \prod_{j=1}^{\ell(\mathbf{r})} \frac{1}{(2r_j + 1)! (4^{R_j} - 1)}. \quad (16)$$

In particular, all coefficients are positive rational numbers.

Proof. Expand the product (13). To obtain total degree $2k$, choose ℓ occupied scale positions $1 \leq j_1 < \dots < j_\ell$ and positive degrees $r_1 + \dots + r_\ell = k$. Their contribution is

$$\prod_{i=1}^{\ell} \frac{4^{-j_i r_i}}{(2r_i + 1)!}.$$

The coefficients are nonnegative, so regrouping by the occupied positions is legitimate. Write $g_1 = j_1$ and $g_i = j_i - j_{i-1}$ for $i > 1$. All gaps are positive and

$$\sum_{i=1}^{\ell} j_i r_i = \sum_{i=1}^{\ell} g_i R_i.$$

Consequently the entire infinite sum over positions is explicitly

$$\sum_{1 \leq j_1 < \dots < j_\ell} 4^{-\sum j_i r_i} = \prod_{i=1}^{\ell} \sum_{g=1}^{\infty} 4^{-g R_i} = \prod_{i=1}^{\ell} \frac{1}{4^{R_i} - 1}.$$

Only the finite sum over compositions remains. For $k = 0$, the empty selection has weight one. $\square$

Example 4.2 (First coefficients). The two compositions of two give

$$b_1 = \frac{1}{3!(4-1)} = \frac{1}{18}, \quad b_2 = \frac{1}{5!(16-1)} + \frac{1}{3!^2(16-1)(4-1)} = \frac{19}{16200}.$$

The four compositions of three give $b_3 = 583/42865200$. No smaller moment has to be evaluated to compute any of these numbers.

Corollary 4.3 (A positive formula at reciprocal powers). *For $n \geq 0$,*

$$F(2^{-n}) = 2^{-\tau_n} \sum_{k=0}^{\lfloor n/2 \rfloor} \frac{1}{(n-2k)!} \sum_{\mathbf{r} \in C_k} \prod_{j=1}^{\ell(\mathbf{r})} \frac{1}{(2r_j + 1)! (4^{R_j} - 1)}. \quad (17)$$

Every summand is positive.

Proof. Take $a = 1$ in (14) and substitute (16). $\square$

Remark 4.4 (Relation to triangular recurrences). Coefficient comparison in (13) also gives

$$(4^k - 1)b_k = \sum_{j=0}^{k-1} \frac{b_j}{(2(k-j) + 1)!} \quad (k \geq 1). \quad (18)$$

Unrolling this identity produces a path interpretation of the same composition weights. Equation (18) is useful as an independent check, but it is not part of the evaluation prescription (16). In particular, the latter is not a recurrence disguised by a new name.

5 Multiplicity partitions and explicit cumulants

5.1 A compact partition formula

Define, for $r \geq 1$,

$$\beta_r = \frac{4^r B_{2r}}{(4^r - 1) 2r (2r)!}, \quad (19)$$

where the Bernoulli convention is $B_1 = -1/2$.

Theorem 5.1 (Partition coefficient formula). *For every $k \geq 0$,*

$$b_k = \sum_{(m_1, \dots, m_k) \in \mathcal{P}_k} \prod_{r=1}^k \frac{\beta_r^{m_r}}{m_r!}. \quad (20)$$

Thus (14) with (20) is another recurrence-free dyadic formula. Equivalently, the even cumulant of order $2r$ is $(2r)!\beta_r$, and every odd cumulant vanishes.

Proof. Near zero, differentiation of the logarithm gives

$$\frac{d}{dt} \log \frac{\sinh t}{t} = \coth t - \frac{1}{t} = \sum_{r \geq 1} \frac{4^r B_{2r}}{(2r)!} t^{2r-1}.$$

Integration with zero constant term, followed by the geometric sum over the scales in (13), yields

$$\log M(t) = \sum_{r \geq 1} \beta_r t^{2r}. \quad (21)$$

Indeed, $\sum_{j \geq 1} 4^{-j^r} = 1/(4^r - 1)$. Exponentiating and selecting t^{2k} amounts to choosing m_r copies of $\beta_r t^{2r}$; this gives exactly (20). The same calculation is valid coefficientwise in formal power series, so no analytic evaluation is required. $\square$

The first values are

$$\beta_1 = \frac{1}{18}, \quad \beta_2 = -\frac{1}{2700}, \quad \beta_3 = \frac{1}{178605}, \quad \beta_4 = -\frac{1}{9639000}. \quad (22)$$

For example, $b_2 = \beta_2 + \beta_1^2/2$, and $b_3 = \beta_3 + \beta_1\beta_2 + \beta_1^3/6$.

5.2 Removing even the Bernoulli symbols

Bernoulli numbers need not be accepted as auxiliary special constants. They have the finite formula

$$B_m = \sum_{v=0}^m \frac{1}{v+1} \sum_{u=0}^v (-1)^u \binom{v}{u} u^m. \quad (23)$$

To verify it, take exponential generating functions. The right-hand side sums formally to

$$\sum_{v \geq 0} \frac{(1 - e^t)^v}{v+1} = \frac{t}{e^t - 1}.$$

Only $v \leq m$ can contribute to the coefficient of t^m , which proves (23) as a finite identity.

There is also a Bernoulli-free formula that uses only factorials:

$$\beta_r = \frac{1}{4^r - 1} \sum_{\ell=1}^r \frac{(-1)^{\ell-1}}{\ell} \sum_{\substack{s_1, \dots, s_\ell \geq 1 \\ s_1 + \dots + s_\ell = r}} \prod_{j=1}^{\ell} \frac{1}{(2s_j + 1)!}. \quad (24)$$

This follows by expanding $\log(1 + \sum_{s \geq 1} t^{2s}/(2s+1)!)$, then summing the scales. Either (23) or (24) makes (20) literally a nested finite rational sum.

5.3 Why this is a different useful representation

The composition formula has 2^{k-1} terms for $k \geq 1$. The partition formula has one term per integer partition of k , and is often much shorter. The price is cancellation: the composition terms are all positive, whereas the β_r alternate in sign. Both are exact rational formulas; neither requires computing $b_0, \dots, b_{k-1}$ before b_k .

For later use, the reciprocal coefficients are equally explicit:

$$\frac{1}{M(t)} = \sum_{r \geq 0} \alpha_r t^{2r}, \quad \alpha_r = \sum_{\mathbf{m} \in \mathcal{P}_r} \prod_{j=1}^r \frac{(-\beta_j)^{m_j}}{m_j!}. \quad (25)$$

For example, $\alpha_0 = 1$, $\alpha_1 = -1/18$, $\alpha_2 = 31/16200$, and $\alpha_3 = -2347/42865200$. The reciprocal series is used only locally at zero or on finite-degree polynomials; no assertion of its being entire is intended.

6 Binary compression of the argument-dependent sum

The master formula has a Thue–Morse prefix of length a . This is avoidable: complete binary blocks can be summed algebraically before any rational arithmetic is done. The resulting identity is useful independently of which closed form is chosen for the coefficients b_k .

For each $i \geq 0$, define the following explicit integer quantities:

$$a_i = \left\lfloor a/2^i \right\rfloor - 2 \left\lfloor a/2^{i+1} \right\rfloor, \quad K_i = 2^i + 2(a \bmod 2^i), \quad \eta_i = \varepsilon_{\lfloor a/2^{i+1} \rfloor}. \quad (26)$$

Here $a_i \in \{0, 1\}$ is the i -th binary digit. In particular, $a \bmod 1 = 0$, so $K_0 = 1$.

Theorem 6.1 (Binary closed form). *For every $a, n \geq 0$, including numerators larger than 2^n ,*

$$\Phi\left(\frac{a}{2^n}\right) = 2^{-\tau_n} \sum_{i=0}^n a_i \eta_i 2^{\tau_i} \sum_{k=0}^{\lfloor (n-i)/2 \rfloor} \frac{b_k 4^{ik} K_i^{n-i-2k}}{(n-i-2k)!}. \quad (27)$$

The coefficients b_k may be supplied by either (16) or (20). Thus the formula is nonrecursive, and its outer sum has at most $n+1$ nonzero positions, independently of the magnitude of a .

Proof. In exponential generating form, the master formula is

$$\Phi(a/2^n) = 2^{-\tau_n} [t^n] M(t) \sum_{h=0}^{a-1} \varepsilon_h e^{(2a-2h-1)t}. \quad (28)$$

For each set bit $a_i = 1$, put $P_i = 2^{i+1} \lfloor a/2^{i+1} \rfloor$. The disjoint blocks

$$\{P_i, P_i + 1, \dots, P_i + 2^i - 1\} \quad (a_i = 1)$$

partition $\{0, \dots, a-1\}$. There are no binary carries when a number $r < 2^i$ is added to P_i , and hence $\varepsilon_{P_i+r} = \eta_i \varepsilon_r$. Also,

$$\sum_{r=0}^{2^i-1} \varepsilon_r e^{-2rt} = \prod_{j=0}^{i-1} (1 - e^{-2^{j+1}t}). \quad (29)$$

Iteration of $M(2t) = (\sinh(t)/t)M(t)$ now gives the exact identity

$$M(t) \prod_{j=0}^{i-1} (1 - e^{-2^{j+1}t}) = 2^{\tau_i} t^i e^{-(2^i-1)t} M(2^i t). \quad (30)$$

Indeed, each factor on the left is $2e^{-2^j t} \sinh(2^j t)$; their powers of two multiply to $2^{i+i(i-1)/2} = 2^{\tau_i}$.

The contribution of the i -th block to (28) is consequently

$$\eta_i 2^{\tau_i} t^i e^{(2a-2P_i-2^i)t} M(2^i t).$$

When $a_i = 1$, write $a = P_i + 2^i + (a \bmod 2^i)$; the exponent becomes $K_i t$. Expanding $M(2^i t) = \sum_{k \geq 0} b_k 4^{ik} t^{2k}$ and extracting t^n proves (27). A block with $i > n$ contains a factor t^i and contributes zero. This explains why even higher binary digits are needed only through the explicit signs η_i . $\square$

6.1 One formula with no unresolved coefficient symbols

Combining (27) and (16) gives the following literal finite expression:

$$\Phi\left(\frac{a}{2^n}\right) = \sum_{i=0}^n a_i \eta_i 2^{\tau_i - \tau_n} \sum_{k=0}^{\lfloor (n-i)/2 \rfloor} \frac{4^{ik} K_i^{n-i-2k}}{(n-i-2k)!} \times \sum_{\ell=0}^k \sum_{\substack{r_1, \dots, r_\ell \geq 1 \\ r_1 + \dots + r_\ell = k}} \prod_{j=1}^{\ell} \frac{1}{(2r_j + 1)! (4^{r_j + \dots + r_\ell} - 1)}. \quad (31)$$

The $\ell = 0$ inner sum is one for $k = 0$, and zero otherwise. Formula (31), together with the digit definitions (26), contains no recursively defined constants at all. It answers the closed-form requirement using only integer arithmetic, factorials, and bounded sums and products.

There are at most

$$\sum_{i=0}^n (\lfloor (n-i)/2 \rfloor + 1) = (d+1)(n-d+1)$$

argument-dependent monomials after the universal coefficients have been computed. This count concerns rational-arithmetic operations, not the bit complexity of multiplying large numerators and denominators.

Example 6.2 (A two-block evaluation). Take $a = 3$, $n = 3$. The set bits are $i = 0, 1$. Their data are $(\eta_0, K_0) = (-1, 1)$ and $(\eta_1, K_1) = (1, 4)$. Therefore

$$F(3/8) = \frac{1}{64} \left[-\left(\frac{1}{6} + \frac{1}{18}\right) + 2 \left(\frac{4^2}{2} + \frac{4}{18}\right) \right] = \frac{73}{288}.$$

For $a = 1$, only $i = 0$ survives, recovering the positive endpoint formula (17).

7 Finite uniform prefixes and a moment-free multinomial formula

This section removes the moments by a different device: replace an infinite uniform sum by several *finite* sums, and extract a polynomial's constant term. No limit remains in the resulting evaluation formula.

7.1 The explicit finite-coordinate polynomial

For $N \geq 0$, set

$$Y_N = \sum_{j=1}^N 2^{-j} V_j, \quad M_N(t) = \mathbb{E} e^{tY_N} = \frac{M(t)}{M(2^{-N}t)},$$

with $Y_0 = 0$. Define

$$\begin{aligned} C_{n,N}(A) &= \frac{1}{n!} \mathbb{E}(A + Y_N)^n \\ &= \sum_{\substack{r_0, r_1, \dots, r_N \geq 0 \\ r_0 + 2r_1 + \dots + 2r_N = n}} \frac{A^{r_0}}{r_0!} \prod_{j=1}^N \frac{4^{-jr_j}}{(2r_j + 1)!}. \end{aligned} \quad (32)$$

Thus $C_{n,0}(A) = A^n/n!$. Every summation index in (32) is bounded by n , and the formula is a finite rational sum whenever A is rational.

For the proof, temporarily write $C_n(A) = [t^n]e^{At}M(t)$. Using the explicit reciprocal coefficients α_r from (25) gives

$$C_{n,N}(A) = \sum_{r=0}^d \alpha_r 4^{-Nr} C_{n-2r}(A). \quad (33)$$

Consequently, $C_{n,N}(A)$ is the value at $z = 4^{-N}$ of a polynomial in z of degree at most $d = \lfloor n/2 \rfloor$, whose constant term is $C_n(A)$. The infinite series in this reasoning has been truncated by coefficient extraction, not approximated.

7.2 Explicit Gaussian weights

For $0 < q < 1$, use

$$(q; q)_j = \prod_{r=1}^j (1 - q^r), \quad \left[\begin{matrix} d \\ j \end{matrix} \right]_q = \frac{(q; q)_d}{(q; q)_j (q; q)_{d-j}},$$

and define

$$\lambda_{d,j}(q) = \frac{(-1)^{d-j} q^{\tau_{d-j}}}{(q; q)_j (q; q)_{d-j}} = \frac{(-1)^{d-j} q^{\tau_{d-j}}}{(q; q)_d} \left[\begin{matrix} d \\ j \end{matrix} \right]_q, \quad 0 \leq j \leq d. \quad (34)$$

In particular, all these weights are rational when q is rational.

Lemma 7.1 (Finite constant-term extraction). *For every polynomial P of degree at most d , and every $z_0 \neq 0$,*

$$P(0) = \sum_{j=0}^d \lambda_{d,j}(q) P(z_0 q^j). \quad (35)$$

Equivalently,

$$\sum_{j=0}^d \lambda_{d,j}(q) q^{jr} = \begin{cases} 1, & r = 0, \\ 0, & 1 \leq r \leq d. \end{cases} \quad (36)$$

Proof. The value at zero of the j -th Lagrange cardinal polynomial for the nodes $z_0, z_0 q, \dots, z_0 q^d$ is

$$\prod_{\substack{0 \leq \ell \leq d \\ \ell \neq j}} \frac{-q^\ell}{q^j - q^\ell}.$$

For $\ell < j$, the factor is $1/(1 - q^{j-\ell})$; for $\ell > j$, it is $-q^{\ell-j}/(1 - q^{\ell-j})$. Multiplication gives exactly (34). Lagrange interpolation proves (35), and applying it to the monomials gives (36). $\square$

Theorem 7.2 (Finite-cube closed form). *Set $Q = 1/4$ and $d = \lfloor n/2 \rfloor$. For every $a, n \geq 0$,*

$$\Phi\left(\frac{a}{2^n}\right) = 2^{-\tau_n} \sum_{h=0}^{a-1} \varepsilon_h \sum_{j=0}^d \lambda_{d,j}(Q) C_{n,j}(2a - 2h - 1), \quad (37)$$

where $C_{n,j}$ is the finite multinomial sum (32). No moment of an infinite distribution is needed in the formula's evaluation.

Proof. Apply (35) with $z_0 = 1$ to the degree- d polynomial in (33), obtaining $C_n(A) = \sum_{j=0}^d \lambda_{d,j}(Q) C_{n,j}(A)$. Substitute this equality into (28). $\square$

There is also a useful coefficient-only version:

$$b_k = \sum_{j=0}^k \lambda_{k,j}(1/4) \sum_{\substack{r_1, \dots, r_j \geq 0 \\ r_1 + \dots + r_j = k}} \prod_{i=1}^j \frac{4^{-ir_i}}{(2r_i + 1)!}. \quad (38)$$

The zero-coordinate inner sum is one for $k = 0$, and zero otherwise. This is a third fully explicit formula for the universal coefficients, with neither ordered compositions nor Bernoulli numbers. A common row of degree $d \geq k$ may replace the row of degree k in (38); the underlying polynomial has degree at most k . Substitution in (27) avoids the long h -sum in (37).

7.3 Arbitrary finite prefixes and other bases

The consecutive depths $0, 1, \dots, d$ are convenient, not compulsory. For distinct nonnegative integers $N_0, \dots, N_d$, put $z_j = 4^{-N_j}$. Then

$$C_n(A) = \sum_{j=0}^d \left(\prod_{\substack{0 \leq \ell \leq d \\ \ell \neq j}} \frac{-z_\ell}{z_j - z_\ell} \right) C_{n, N_j}(A). \quad (39)$$

If $N_j = N + Lj$ with integers $N \geq 0$, $L \geq 1$, these weights reduce to $\lambda_{d,j}(4^{-L})$. Thus the quarter-base formula belongs to a whole family of exact finite rational filters.

8 An exact quarter-base spline formula with no moments

The previous construction used moments of finite uniform prefixes. Their *densities* give a particularly compact double-sum answer. The key fact is an exact polynomial dependence on 4^{-N} at a fixed dyadic point.

8.1 The finite spline in explicit form

For $N \geq 1$, let p_N be the density of Y_N , and put $S_N(x) = p_N(x - 1)$. The first N Fourier factors are

$$\widehat{p}_N(\xi) = \prod_{k=0}^{N-1} \frac{\sin(\pi\xi/2^k)}{\pi\xi/2^k}.$$

Thus a convention that calls the product ending at $k = m$ by up_m has $p_N = \text{up}_{N-1}$. This one-factor indexing shift matters in exact formulas.

Convolution of the N uniform densities, or direct inclusion–exclusion for a weighted cube, gives

$$S_N(x) = \frac{2^{N(N-1)/2}}{(N-1)!} \sum_{h=0}^{2^N-1} \varepsilon_h \left(x - \frac{2h+1}{2^N} \right)_+^{N-1}. \quad (40)$$

For exponent zero, t_+^0 means the indicator of $t > 0$; endpoint values of individual densities are immaterial. No point used below is a knot. For completeness, the j -th centered uniform has

width 2^{1-j} . The product of their reciprocal widths is $2^{N(N-1)/2}$, and their subset shifts, after translation by one, are exactly $(2h+1)/2^N$, with inclusion–exclusion sign ε_h . This proves the normalization in (40). Finite-convolution formulas of this type underlie the construction discussed by Pinelis [5]; the extraction below is what removes the limiting step.

At a dyadic point there is an especially simple integer-power form. For $0 \leq a \leq 2^n$ and $N \geq n+1$,

$$S_N\left(\frac{a}{2^n}\right) = \frac{1}{2^{N(N-1)/2}(N-1)!} \sum_{h=0}^{a2^{N-n-1}-1} \varepsilon_h (a2^{N-n} - 2h - 1)^{N-1}. \quad (41)$$

Indeed, $a2^{N-n}$ is even, so none of the parentheses vanishes. Precisely the first $a2^{N-n-1}$ terms in (40) are active. Factoring out $2^{-N(N-1)}$ produces the displayed power of two.

8.2 Why the scale dependence is a finite polynomial

Lemma 8.1 (Degree reduction on a dyadic spline cell). *Fix $x = a/2^n$, where $0 \leq a \leq 2^n$, and let $N \geq n+1$. On the open interval $|t| < 2^{-N}$, the function $S_N(x+t)$ is a polynomial $P_N(t)$ of degree at most n , even though a general piece of S_N has nominal degree $N-1$.*

Proof. Write $N = n+1+L$, with $L \geq 0$. The point x is the midpoint between consecutive knots, and the active indices throughout this cell are $0 \leq h < a2^L$. Split them as $h = b2^L + r$, where $0 \leq b < a$, $0 \leq r < 2^L$. Their signs factor as $\varepsilon_h = \varepsilon_b \varepsilon_r$.

The finite product (29) has a zero of order L at zero. Differentiating it shows the Prouhet cancellation

$$\sum_{r=0}^{2^L-1} \varepsilon_r r^j = 0 \quad (0 \leq j < L). \quad (42)$$

Each block of the active spline expression is a signed sum of powers $(A+t-Br)^{n+L}$, for fixed A, B . The coefficient of t^m is a polynomial in r of degree at most $n+L-m$. It vanishes after summation when $m > n$, by (42). Each block, and hence P_N , has degree at most n . The case $L=0$ requires no cancellations. $\square$

Theorem 8.2 (Exact finite scale law). *With the same hypotheses, and $d = \lfloor n/2 \rfloor$,*

$$S_N(x) = \sum_{r=0}^d \alpha_r 4^{-Nr} \Phi^{(2r)}(x) = F(x) + \sum_{r=1}^d \alpha_r \Phi^{(2r)}(x) 4^{-Nr}. \quad (43)$$

In particular, the sampled spline value is an exact polynomial in 4^{-N} of degree at most d , whose constant coefficient is $F(x)$.

Proof. The independent tail decomposition is

$$Y \stackrel{\text{law}}{=} Y_N + 2^{-N} Y',$$

where Y' has the law of Y . Therefore $\text{up} = p_N * \text{law}(2^{-N} Y')$. Near the points in question the signed function is $\Phi(x) = \text{up}(x-1)$; this also identifies all endpoint derivatives. For derivative orders $0 \leq s \leq n \leq N-1$, differentiation of the convolution is valid using the locally integrable derivatives of p_N . There are no atomic terms at these derivative orders: p_N is a spline of order N . Almost every tail argument lies strictly within the open spline cell of the preceding lemma. It follows that

$$\Phi^{(s)}(x) = \mathbb{E} P_N^{(s)}(-2^{-N} Y'), \quad 0 \leq s \leq n. \quad (44)$$

This argument includes $N = 1$, when only the undifferentiated density is used.

Let D denote differentiation of polynomials in t , and let

$$J_x(t) = \sum_{s=0}^n \frac{\Phi^{(s)}(x)}{s!} t^s$$

be the *finite Taylor jet*. Since P_N has degree at most n , (44) is the polynomial identity

$$J_x(t) = \mathbb{E}P_N(t - 2^{-N}Y') = M(2^{-N}D)P_N(t). \quad (45)$$

Evenness of M removes the minus sign. On polynomials of degree at most n , $D^{n+1} = 0$; thus the reciprocal operator is the finite sum

$$M(2^{-N}D)^{-1} = \sum_{r=0}^d \alpha_r 4^{-Nr} D^{2r}.$$

Applying it to (45) and evaluating at $t = 0$ proves (43). $\square$

Remark 8.3 (A necessary distinction). The proof does *not* assert that $F(x+t) = J_x(t)$ on an interval. The polynomial in the proof is a piece of the *finite spline*, and only finitely many derivatives of the convolution at the single point x are used. Replacing the smooth Fabius function by its finite Taylor jet on a neighborhood would not be justified.

8.3 The closed double sum

Theorem 8.4 (Quarter-base dyadic formula). *For $n \geq 0$, $0 \leq a \leq 2^n$, $d = \lfloor n/2 \rfloor$, and $Q = 1/4$,*

$$F(a/2^n) = \sum_{j=0}^d \lambda_{d,j}(Q) S_{n+1+j}(a/2^n). \quad (46)$$

Equivalently, the following is a closed form containing no moments or derivatives:

$$\boxed{F\left(\frac{a}{2^n}\right) = \frac{1}{(Q; Q)_d} \sum_{j=0}^d \frac{(-1)^{d-j} Q^{\tau_{d-j}}}{2^{\tau_{n+j}} (n+j)!} \begin{bmatrix} d \\ j \end{bmatrix}_Q} \quad (47)$$

$$\times \sum_{h=0}^{2^j a - 1} \varepsilon_h (2^{j+1}a - 2h - 1)^{n+j}, \quad Q = \frac{1}{4}.$$

Proof. Apply (35) to the polynomial in (43) at the $d+1$ nodes $4^{-(n+1)}, 4^{-(n+2)}, \dots, 4^{-(n+1+d)}$. Its value at zero is $F(a/2^n)$. This gives (46). Substituting (41) with $N = n+1+j$, and then (34), proves (47). $\square$

Formula (47) uses only $d+1$ splines, of orders $n+1, \dots, n+d+1$. Its largest power is $n+d$, and the total number of signed powers before simplification is

$$\sum_{j=0}^d 2^j a = a(2^{d+1} - 1). \quad (48)$$

This formula is attractive when the absence of all auxiliary coefficient sequences is more important than minimizing arithmetic. For large numerators, the binary formula is usually the better algebraic organization.

8.4 Other prefix depths, conditioning, and the half-base comparison

For any integers $N \geq n + 1$, $L \geq 1$, the same proof gives

$$F(a/2^n) = \sum_{j=0}^d \lambda_{d,j}(4^{-L}) S_{N+Lj}(a/2^n). \quad (49)$$

Arbitrary distinct depths at least $n + 1$ are also allowed, using the Lagrange weights in (39). There is no convergence parameter to send to infinity in any of these identities.

The finite q -binomial theorem gives the exact absolute row sum

$$\sum_{j=0}^d |\lambda_{d,j}(q)| = \frac{(-q; q)_d}{(q; q)_d} = \prod_{r=1}^d \frac{1 + q^r}{1 - q^r}. \quad (50)$$

For example, this is bounded uniformly in d by $\exp(2q/(1 - q)^2)$. Indeed, $\log((1 + x)/(1 - x)) \leq 2x/(1 - x)$ for $0 \leq x < 1$, and the resulting geometric sum gives the bound. This measures only amplification by the *outer filter*; it does not eliminate cancellation inside the large signed power sums.

The exact polynomial in (43) is also a polynomial of degree at most $2d \leq n$ in 2^{-N} . Hence the longer half-base row

$$F(a/2^n) = \sum_{j=0}^n \lambda_{n,j}(1/2) S_{n+1+j}(a/2^n) \quad (51)$$

is valid too. Inserting the spline formula and rewriting its power as $(-2)^{n+j}(h - 2^j a + 1/2)^{n+j}$ gives exactly the half-base form

$$\frac{1}{2^{n^2}(1/2; 1/2)_n} \sum_{j=0}^n \frac{[n]_{1/2}}{2^{j(j-1)}(n+j)!} \sum_{h=0}^{2^j a - 1} \varepsilon_h (h - 2^j a + 1/2)^{n+j}. \quad (52)$$

This is the formula appearing in Reshetnikov’s 2019 question [4]. The quarter-base row exploits the absence of odd powers of 2^{-N} and removes unnecessary samples. The underlying interpolation principle and related quarter-base constructions are already part of the repository’s broader development [1]; the point here is the explicit, recurrence-free form and a self-contained exactness proof.

8.5 When the degree is sharp

The degree bound d is attained at every reduced interior dyadic point. We record a proof to distinguish a genuinely finite error polynomial from a merely convenient upper bound.

First,

$$(-1)^r \alpha_r > 0 \quad (r \geq 0). \quad (53)$$

For example, the Euler product for the sine or hyperbolic sine [6, §4.36] gives, locally at zero,

$$\frac{1}{M(iz)} = \prod_{j \geq 1} \prod_{m \geq 1} \left(1 - \frac{z^2}{4^j \pi^2 m^2} \right)^{-1}.$$

All coefficients of this product in z^2 are strictly positive. Comparison with $\sum_r (-1)^r \alpha_r z^{2r}$ proves (53). The products converge locally, so coefficient comparison is legitimate.

Now take $n \geq 1$, $0 < a < 2^n$, and a odd. For $0 \leq 2r \leq n$,

$$\Phi^{(2r)}(a/2^n) = 2^{72r} \Phi(a/2^{n-2r}) \neq 0.$$

The argument on the right is not an even integer; each open bump block has a constant nonzero sign and positive absolute value. Positivity follows also from (4): every open subinterval of the support has positive probability. Thus every coefficient in (43), including its highest one, is nonzero.

Accordingly, $d + 1$ is the required sample count for a universal linear rule extracting the constant term from an arbitrary degree- d polynomial at distinct nonzero nodes. With only d nodes, their vanishing polynomial has degree d but a nonzero constant term. This is a statement about universal interpolation, not a lower bound against every possible special-purpose formula for one value.

9 An integer determinant and its explicit permutation sum

The composition and partition formulas expose individual coefficients. A bordered matrix packages the entire dyadic value into a single ratio of integers. The determinant can then be expanded into a finite sum of products, so no matrix inversion or recurrence is part of the mathematical definition.

For $d = \lfloor n/2 \rfloor$, form the $(d + 1) \times (d + 1)$ lower-triangular integer matrix $T = (T_{ij})_{0 \leq i, j \leq d}$ by

$$T_{00} = 1, \quad T_{ii} = (2i + 1)(4^i - 1) \quad (i \geq 1), \quad T_{ij} = -\binom{2i + 1}{2j} \quad (j < i), \quad (54)$$

with $T_{ij} = 0$ for $j > i$. Define the explicitly known column vectors

$$e_0 = (1, 0, \dots, 0)^\top, \quad v_k = \binom{n}{2k} \sum_{h=0}^{a-1} \varepsilon_h (2a - 2h - 1)^{n-2k} \quad (0 \leq k \leq d), \quad (55)$$

and the $(d + 2) \times (d + 2)$ bordered matrix

$$A = \begin{pmatrix} T & e_0 \\ v^\top & 0 \end{pmatrix}. \quad (56)$$

Every entry of A is an explicit integer.

Theorem 9.1 (Bordered determinant closed form). *For every $a, n \geq 0$,*

$$\boxed{\Phi\left(\frac{a}{2^n}\right) = -\frac{\det A}{2^{\tau_n} n! \prod_{i=1}^d (2i + 1)(4^i - 1)}}. \quad (57)$$

In particular, this is a rational closed form without unspecified auxiliary values.

Proof. The moment identity (18), after multiplying by $(2i + 1)!$, becomes

$$(2i + 1)(4^i - 1)c_i = \sum_{j=0}^{i-1} \binom{2i + 1}{2j} c_j, \quad c_0 = 1.$$

Consequently, $Tc = e_0$, where $c = (c_0, \dots, c_d)^\top$. This is used only to prove the formula, not to define its evaluation. Since T is triangular,

$$\det T = \prod_{i=1}^d (2i + 1)(4^i - 1) \neq 0.$$

The Schur-complement identity gives $\det A = -\det T v^\top T^{-1} e_0 = -\det T v^\top c$. Finally, (14) states $\Phi(a/2^n) = 2^{-\tau_n} v^\top c / n!$. $\square$

A determinant need not be treated as an extra permitted primitive. With indices $0, \dots, d+1$, its Leibniz expansion is

$$\det A = \sum_{\sigma \in \mathfrak{S}_{d+2}} (-1)^{\#\{(i,j): i < j, \sigma(i) > \sigma(j)\}} \prod_{i=0}^{d+1} A_{i, \sigma(i)}. \quad (58)$$

Equations (54)–(58) are therefore a finite sum/product answer in exactly the requested sense. Many terms vanish because T is triangular. For numerical implementation, an exact determinant algorithm is preferable to literally enumerating all permutations; the formula’s closed-form status does not require the least efficient evaluation order.

Corollary 9.2 (An explicit common denominator). *Set*

$$D_n = 2^{\tau_n} n! \prod_{i=1}^{\lfloor n/2 \rfloor} (2i+1)(4^i - 1). \quad (59)$$

Then $D_n \Phi(a/2^n)$ is an integer for every nonnegative integer a . The same denominator works for all Rvachev values on the grid of spacing 2^{-n} .

This is a convenient denominator, not a claim of minimality or a replacement for the sharper arithmetic questions studied in [2].

10 Domains, normalizations, and dyadic derivatives

10.1 Bounded Fabius and compact Rvachev values

The binary, composition, cube, and determinant formulas above are stated for $\Phi(a/2^n)$. To obtain the bounded distribution F , use the same formulas for $0 \leq a \leq 2^n$, and use

$$F(a/2^n) = 0 \quad (a \leq 0), \quad F(a/2^n) = 1 \quad (a \geq 2^n). \quad (60)$$

The boundary values agree with both prescriptions.

For any integer m , the compactly supported Rvachev value is

$$\boxed{\text{up}(m/2^n) = \begin{cases} F((2^n - |m|)/2^n), & |m| \leq 2^n, \\ 0, & |m| > 2^n. \end{cases}} \quad (61)$$

Thus every formula for bounded F immediately gives a closed rational formula for up , with no new coefficients. In particular, $\text{up}(0) = 1$, $\text{up}(\pm 1) = 0$, and $\text{up}(\pm 1/2) = 1/2$. One can also apply the spline filter directly to the centered densities: for $|m| \leq 2^n$,

$$\text{up}(m/2^n) = \sum_{j=0}^d \lambda_{d,j}(1/4) p_{n+1+j}(m/2^n).$$

This follows from evenness and (46), or from the same local spline-cell proof.

10.2 Signed global values and nonreduced fractions

For $x \geq 0$, let $b = \lfloor x/2 \rfloor$. The fold

$$\Phi(x) = \varepsilon_b F(1 - |x - 2b - 1|) \quad (62)$$

reduces every signed-global value to a bounded one. At shared endpoints the right-hand side is zero, so the choice of adjacent block has no effect. For $x < 0$, $\Phi(x) = 0$. Formula (27) performs this reduction implicitly through the high-bit signs and works for every nonnegative numerator without a separate fold. In particular,

$$\Phi(2b) = 0, \quad \Phi(2b + 1) = \varepsilon_b. \quad (63)$$

For instance, $F(3) = 1$, whereas $\Phi(3) = -1$. The distinction is essential when an argument lies outside $[0, 1]$.

No result requires $a/2^n$ to be reduced. Directly from the proved identities,

$$\mathcal{F}_n(a) = \mathcal{F}_{n+1}(2a),$$

where $\mathcal{F}_n(a)$ denotes any of the appropriate closed expressions for the same function. Reducing the denominator merely shortens the sums. For bounded F , the symmetry $F(a/2^n) + F((2^n - a)/2^n) = 1$ gives another useful reduction.

10.3 Closed forms for every dyadic derivative

The value formulas immediately cover all derivatives. For $a, n \geq 0$,

$$\Phi^{(r)}(a/2^n) = \begin{cases} 2^{\tau_r} \Phi(a/2^{n-r}), & 0 \leq r \leq n, \\ 0, & r \geq n + 1. \end{cases} \quad (64)$$

For $r \leq n$, this is (10) with an explicit dyadic argument. For $r \geq n + 1$, the argument $2^r a/2^n$ is an even integer, so (63) gives zero.

On the interior of $[0, 1]$, the derivatives of bounded F agree with those of Φ . At 0 and 1, all positive-order derivatives of bounded F vanish. For Rvachev derivatives, use the signed extension without an absolute-value fold:

$$\text{up}^{(r)}(m/2^n) = 2^{\tau_r} \Phi\left(\frac{m + 2^n}{2^{n-r}}\right) \quad (|m| \leq 2^n, \ 0 \leq r \leq n), \quad (65)$$

and all higher derivatives vanish at these dyadic points. Outside the support all derivatives vanish too. Formula (65) includes the endpoint flatness; it follows from $\text{up}(x) = \Phi(x + 1)$ on $[-1, 1]$. Thus every entry of every dyadic Taylor jet is explicitly rational. As emphasized in Section 8, this does not make that jet equal to the function on an interval.

11 Exact examples and independent arithmetic checks

11.1 Universal coefficients and a dyadic table

The first even moments, with the normalization $c_k = (2k)!b_k$, are

k	b_k	c_k
0	1	1
1	1/18	1/9
2	19/16200	19/675
3	583/42865200	583/59535
4	132809/1311675120000	132809/32531625

These numbers result independently from the composition, partition, and finite-prefix formulas. They are universal: the dyadic numerator changes only the outer polynomial evaluation.

For a compact table of function values, all sixteenths up to the midpoint are

a	$F(a/16)$	a	$F(a/16)$
0	0	5	305857/2073600
1	143/2073600	6	73/288
2	1/288	7	777743/2073600
3	46657/2073600	8	1/2
4	5/72		

The remaining values follow from reflection. For example, $\text{up}(11/16) = F(5/16) = 305857/2073600$.

11.2 Two-point and three-point spline extraction

For $d = 1$, the quarter-base weights are

$$(\lambda_{1,0}, \lambda_{1,1}) = (-1/3, 4/3).$$

At $x = 1/4$, $n = 2$, the two finite splines are $S_3(x) = 1/16$ and $S_4(x) = 13/192$. Therefore

$$F(1/4) = -\frac{1}{3} \frac{1}{16} + \frac{4}{3} \frac{13}{192} = \frac{5}{72}.$$

Likewise, at $x = 3/8$,

$$S_4(x) = \frac{97}{384}, \quad S_5(x) = \frac{389}{1536}, \quad F(3/8) = \frac{4S_5(x) - S_4(x)}{3} = \frac{73}{288}.$$

These are exact equalities, not numerical extrapolations.

For $d = 2$,

$$(\lambda_{2,0}, \lambda_{2,1}, \lambda_{2,2}) = (1/45, -4/9, 64/45).$$

At $x = 5/16$, the relevant values are

$$S_5(x) = \frac{1205}{8192}, \quad S_6(x) = \frac{289799}{1966080}, \quad S_7(x) = \frac{13917509}{94371840}.$$

Their exact weighted sum is

$$\frac{1}{45} S_5(x) - \frac{4}{9} S_6(x) + \frac{64}{45} S_7(x) = \frac{305857}{2073600}.$$

11.3 Examples with small values and larger denominators

The positive formula avoids subtracting large alternating terms when $a = 1$. For example,

$$F(1/1024) = \frac{134926369}{1246394851358539387238350848000} \approx 1.082533106205739 \cdot 10^{-22}.$$

The binary formula also gives, without a prefix sum of length 173,

$$F(173/4096) = \frac{1490588447599319294372326859976672988757}{292214732887898713986916575925267070976000000} \approx 5.101003747717089 \cdot 10^{-6}. \quad (66)$$

Only the five set bits of $173 = 128 + 32 + 8 + 4 + 1$ contribute to (27). The decimal displays are for orientation; the fractions are the actual outputs of the exact formulas.

11.4 What was checked computationally

The accompanying standard-library Python program performs rational arithmetic with `Fractions.Fraction`; no floating-point comparison is used in its assertions. It independently implements the positive composition formula, the cumulant partition formula with finite Bernoulli sums, the finite-prefix formula, the Thue–Morse master formula, the binary formula, the quarter-base spline formula, and the determinant. The recorded run verifies the following specific ranges.

The three coefficient calculations agree for $0 \leq k \leq 12$, where the third comparison is a separately isolated moment-recurrence oracle. Finite-prefix coefficients agree for $0 \leq k \leq 6$. The reciprocal coefficients satisfy the exact convolution identity for $1/M$, and the Gaussian weights satisfy (36) through $d = 6$.

For every $0 \leq n \leq 9$ and $0 \leq a \leq 2^n$, the binary, Thue–Morse, spline, and finite-cube evaluations agree. These are 1033 dyadic *representations*, including nonreduced ones, rather than 1033 distinct rational arguments. Refinement $(a, n) \mapsto (2a, n + 1)$ and reflection are checked for the same grid. The determinant is compared throughout $0 \leq n \leq 6$.

A separate test covers all 388 representations with $0 \leq n \leq 6$ and $0 \leq a \leq 3 \cdot 2^n$, comparing the binary formula, the master formula, and the signed global fold. Further cases check nonconsecutive spline depths, larger starting depths, and the actual polynomial coefficients in (43). All assertions pass in the supplied output. The computations are checks of the proofs, not substitutes for them. The isolated recurrence oracle is not called by any of the closed-form evaluation routines.

12 How the alternatives compare

12.1 Closed form versus evaluation algorithm

An explicit finite formula can be evaluated by loops, memoization, or an exact linear-algebra algorithm without ceasing to be a closed form. What is excluded here is a definition that leaves a desired auxiliary constant specified only in terms of earlier unknown constants. For example, storing previously evaluated b_k ’s is optional caching, whereas defining b_k only by (18) would fail the requested criterion.

The composition sum has 2^{k-1} terms for $k \geq 1$. Computing all coefficients through d by their individual explicit composition sums uses 2^d composition terms in total, including b_0 , and at most d factors per term. The partition representation reduces the number of terms to the number of integer partitions, but introduces signs. The finite-prefix representation uses only weak compositions and rational Gaussian weights. Once coefficients are available, binary evaluation has at most $(d + 1)(n - d + 1)$ polynomial terms.

The spline double sum is the most direct answer with no named coefficient sequence: it uses only signs, powers, factorials, and a Gaussian row. Its explicit $a(2^{d+1} - 1)$ signed powers are a much larger expression for large numerators, and intermediate integers can be enormous. Exact arithmetic is therefore important. The determinant uses a matrix of order $d + 2$ and immediately supplies a common denominator, but its literal permutation expansion is not the preferred numerical algorithm.

None of these observations asserts that a nonrecursive formula must outperform a well-designed recurrence. The purpose is to expose exact finite structure and to provide independent representations.

12.2 Relation to the supplied repository and the literature

The analytic identities, the Thue–Morse moment formula, and much of the underlying coefficient and interpolation machinery already appear in the supplied exposition [1] or the cited literature. The repository also has related path, composition, cumulant, finite-prefix, and quarter-scale developments. Thus it would be misleading to label every formula family in this article as a wholly new invention.

The useful alternative presentations assembled here are specific. The occupied-uniform-scale proof gives an all-positive suffix-composition weight for each normalized moment. The binary-block calculation incorporates the moment transform directly, eliminating the long numerator-dependent prefix. The finite-coordinate formula substitutes explicit finite cube moments for unknown infinite moments. The parity-reduced spline expression is written as a standalone quarter-base double sum, with an exact local-cell proof and a precise factor-count convention. Finally, the integer border converts the moment system into one finite permutation sum for the desired value itself.

These statements describe the derivations and their uses, not a comprehensive priority claim. The linked repository was consulted as a living source on 4 September 2026; no immutable commit identifier was established for that citation. No claim is made that the proofs or the included Python verification have been translated into Lean.

13 Conclusion

Every dyadic value in question admits multiple recurrence-free rational representations. For practical symbolic evaluation, a useful choice is the binary formula (27), with its universal coefficients supplied by the positive composition sum (16) or the partition sum (20). For a single displayed formula containing no coefficient sequence, use (31) or the quarter-base spline double sum (47). The finite-cube formula (37) and the bordered permutation formula (57)–(58) give further independent alternatives.

The common mechanism is finite algebra at a dyadic point: binary cancellation, finite coefficient extraction, or exact interpolation of a polynomial in the tail scale. Infinite products and probability distributions provide the proofs, but neither infinite limits nor auxiliary recurrences survive in the evaluation formulas.

A A compact nonrecursive exact evaluator

The following core implements (16) and (27) using only the Python standard library. Compositions are enumerated by subsets of the $k - 1$ possible cut positions, rather than by recursively generating smaller compositions. The full accompanying file `verify_formulas.py` additionally implements all the other formula families and the checks in Section 11.

```
from fractions import Fraction
from functools import lru_cache
from math import factorial

def compositions(k: int):
    if k < 0:
        raise ValueError("k must be nonnegative")
    if k == 0:
        yield ()
        return
    for mask in range(1 << (k - 1)):
```

```

        cuts = [0] + [j for j in range(1, k)
                       if (mask >> (j - 1)) & 1] + [k]
        yield tuple(v - u for u, v in zip(cuts, cuts[1:]))

@lru_cache(None)
def coefficient(k: int) -> Fraction:
    total = Fraction(0)
    for parts in compositions(k):
        term, suffix = Fraction(1), k
        for r in parts:
            term /= factorial(2*r + 1) * (4**suffix - 1)
            suffix -= r
        total += term
    return total

def signed_dyadic(a: int, n: int) -> Fraction:
    """Signed global Phi(a / 2**n), not the clamped CDF."""
    if not isinstance(a, int) or not isinstance(n, int):
        raise TypeError("a and n must be integers")
    if a < 0 or n < 0:
        raise ValueError("a and n must be nonnegative")
    total = Fraction(0)
    for i in range(n + 1):
        if not ((a >> i) & 1):
            continue
        sign = -1 if (a >> (i + 1)).bit_count() % 2 else 1
        K = (1 << i) + 2 * (a % (1 << i))
        inner = sum(
            (coefficient(k) * 4**(i*k)
             * Fraction(K**(n-i-2*k), factorial(n-i-2*k))
             for k in range((n-i)//2 + 1)), Fraction(0))
        total += sign * 2**(i*(i+1)//2) * inner
    return total / 2**(n*(n+1)//2)

def fabius(x: Fraction) -> Fraction:
    x = Fraction(x)
    if x <= 0:
        return Fraction(0)
    if x >= 1:
        return Fraction(1)
    den = x.denominator
    if den & (den - 1):
        raise ValueError("argument must be dyadic")
    return signed_dyadic(x.numerator, den.bit_length() - 1)

def up(x: Fraction) -> Fraction:
    return fabius(1 - abs(Fraction(x)))

assert fabius(Fraction(5, 16)) == Fraction(305857, 2073600)
assert up(Fraction(11, 16)) == fabius(Fraction(5, 16))
assert signed_dyadic(3, 0) == -1

```

Pass exact rational inputs such as `Fraction(173, 4096)`. Passing a decimal floating-point number instead asks for the exact rational represented by that binary floating-point value, which may have a much larger denominator than intended. The code uses `int.bit_count`, available in Python 3.10 and later.

B Notation and reproducibility checklist

Symbol	Meaning
F	Bounded cumulative distribution, clamped to zero and one.
Φ	Signed global extension satisfying $\Phi' = 2\Phi(2\cdot)$.
up	Even bump of height one and integral one on $[-1, 1]$.
c_k, b_k	$c_k = \mathbb{E}Y^{2k}$, $b_k = c_k/(2k)!$.
β_r, α_r	Coefficients of $\log M(t)$ and $1/M(t)$ in even powers.
τ_n, d	$n(n+1)/2$, $\lfloor n/2 \rfloor$.
ε_h	$(-1)^{s_2(h)}$, defined directly from binary digits.
p_N, S_N	Density of N centered uniforms; $S_N(x) = p_N(x-1)$.
$\lambda_{d,j}(q)$	Explicit Lagrange constant-term weights, (34).

The archive contains this L^AT_EX source, the rendered PDF, the complete Python verification program, its recorded output, and a short README. The L^AT_EX file uses its own bibliography and requires no external .bib file or image assets. It can be built by running `pdflatex` twice, or by `latexmk -pdf`. The verification program runs with `python verify_formulas.py` and requires no third-party packages.

References

- [1] V. Reshetnikov and contributors, *Fabius Function and Rvachev Up*, ProveIt repository, Analysis/FabiusFunction/docs/Fabius_Function_and_Rvachev_Up. Living exposition, consulted 4 September 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Analysis/FabiusFunction/docs/Fabius_Function_and_Rvachev_Up.
- [2] J. Arias de Reyna, *Arithmetic of the Fabius function*, Integers **18** (2018), Paper A51, 20 pp. <https://math.colgate.edu/~integers/s51/s51.pdf>. Preprint: <https://arxiv.org/abs/1702.06487>.
- [3] J. Arias de Reyna, *An infinitely differentiable function with compact support: Definition and properties*, English translation (2017) of the author’s 1982 paper. <https://arxiv.org/abs/1702.05442>.
- [4] V. Reshetnikov, *Conjectured formula for the Fabius function*, Mathematics Stack Exchange, question posted 5 July 2019. <https://math.stackexchange.com/questions/3283519/conjectured-formula-for-the-fabius-function>.
- [5] I. Pinelis, Answer to *A non-recursive, explicit formula for the Fabius function*, MathOverflow, January 2020; also cross-posted to Mathematics Stack Exchange. <https://mathoverflow.net/questions/351004/a-non-recursive-explicit-formula-for-the-fabius-function>.
- [6] NIST Digital Library of Mathematical Functions, §4.36, *Infinite Products and Partial Fractions*. <https://dlmf.nist.gov/4.36>.